## Challenge 1

La ricerca pubblica (/articles/search) è aperta a tutti. Se qualcuno manda tantissime richieste di fila, il server può andare in difficoltà (DoS).

### Attacco riuscito (prima delle mitigazioni)

Ho usato lo script bash nella cartella XXX-AttackTools/dos (versione Mac) puntato a http://cyber.blog:8000/articles/search.

XXX-AttackTools > dos > $ search_manage_mem_mac.sh

```
#!/bin/bash

# URL della rotta da attaccare
URL="URL="http://cyber.blog:8000/articles/search""

# Generare un grande payload casuale
LARGE_PAYLOAD=$(head -c 50000 < /dev/urandom | base64)

# Numero di richieste da inviare
NUM_REQUESTS=100

# Limite massimo di processi simultanei
MAX_PARALLEL_REQUESTS=200

```

```
Richiesta 87 inviata
Richiesta 88 inviata
Richiesta 89 inviata
Richiesta 90 inviata
Richiesta 91 inviata
Richiesta 92 inviata
Richiesta 93 inviata
Richiesta 94 inviata
Richiesta 95 inviata
Richiesta 96 inviata
Richiesta 97 inviata
Richiesta 98 inviata
Richiesta 99 inviata
Richiesta 100 inviata
Attacco DoS simulato completato!
```

Per verificare in modo chiaro ho anche lanciato più richieste di fila con curl: tutte rispondevano **200**, quindi il server accettava tutto senza limiti.

```
=test"
> done
200
200
200
200
200
200
200
200
200
```

## Mitigazione

Ho aggiunto:

- rate limit sulla search (max 5 richieste al minuto per IP) e, se si supera, un blocco temporaneo dell'IP (10 minuti);
- rate limit globale (60 richieste/minuto);
- rate limit anche su /careers/submit;
- ho controllato che Fortify abbia già il limite sul login (5 al minuto).

routes > web.php

```php
use App\Http\Controllers\PublicController;
use App\Http\Controllers\WriterController;
use App\Http\Controllers\ArticleController;
use App\Http\Controllers\RevisorController;

// Public routes
Route::get('/', [PublicController::class, 'homepage'])->name('homepage');
Route::get('/careers', [PublicController::class, 'careers'])->name('careers');
Route::post('/careers/submit', [PublicController::class, 'careersSubmit'])->name('careers.submit');

Route::get('/articles/index', [ArticleController::class, 'index'])->name('articles.index');
Route::get('/articles/show/{article:slug}', [ArticleController::class, 'show'])->name('articles.
show');
Route::get('/articles/category/{category}', [ArticleController::class, 'byCategory'])->name('articles.
byCategory');
Route::get('/articles/user/{user}', [ArticleController::class, 'byUser'])->name('articles.byUser');
Route::get('/articles/search', [ArticleController::class, 'articleSearch'])
    ->middleware('throttle:5,1')
    ->name('articles.search');

// Writer routes
Route::middleware('writer')->group(function(){
    Route::get('/articles/create', [ArticleController::class, 'create'])->name('articles.create');
    Route::post('/articles/store', [ArticleController::class, 'store'])->name('articles.store');
    Route::get('/writer/dashboard', [WriterController::class, 'dashboard'])->name('writer.dashboard');
```

utes > web.php

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\AdminController;
use App\Http\Controllers\PublicController;
use App\Http\Controllers\WriterController;
use App\Http\Controllers\ArticleController;
use App\Http\Controllers\RevisorController;

// Public routes
Route::get('/', [PublicController::class, 'homepage'])->name('homepage');
Route::get('/careers', [PublicController::class, 'careers'])->name('careers');
Route::post('/careers/submit', [PublicController::class, 'careersSubmit'])
    ->middleware('throttle:5,1')
    ->name('careers.submit');
Route::get('/articles/index', [ArticleController::class, 'index'])->name('articles.index');
Route::get('/articles/show/{article:slug}', [ArticleController::class, 'show'])->name('articles.
```

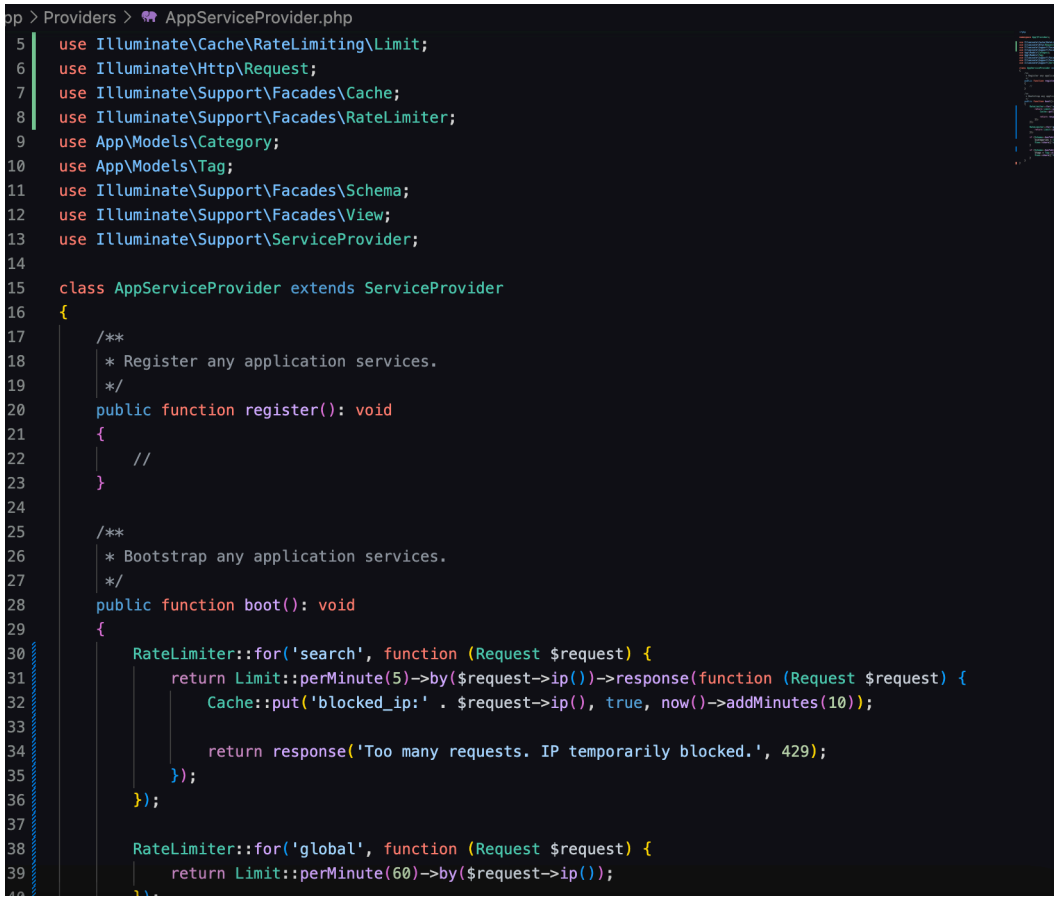
otstrap > app.php

```php
<?php

use App\Http\Middleware\FrameGuard;
//use App\Http\Middleware\RateLimit;
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        $middleware->appendToGroup('web', \App\Http\Middleware\TemporaryIpBlock::class);
        $middleware->appendToGroup('web', 'throttle:global');

        $middleware->alias([
            'admin' => App\Http\Middleware\UserIsAdmin::class,
            'revisor' => App\Http\Middleware\UserIsRevisor::class,
            'writer' => App\Http\Middleware\UserIsWriter::class,
            'admin.local'=> App\Http\Middleware\OnlyLocalAdmin::class
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions) {
        //
    })->create();
```


pp > Providers > AppServiceProvider.php
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Facades\RateLimiter;
use App\Models\Category;
use App\Models\Tag;
use Illuminate\Support\Facades\Schema;
use Illuminate\Support\Facades\View;
use Illuminate\Support\ServiceProvider;
class AppServiceProvider extends ServiceProvider
{
/**
* Register any application services.
*/
public function register(): void
{
//
}
/**
* Bootstrap any application services.
*/
public function boot(): void
{
RateLimiter::for('search', function (Request $request) {
return Limit::perMinute(5)->by($request->ip())->response(function (Request $request) {
Cache::put('blocked_ip:' . $request->ip(), true, now()->addMinutes(10));
return response('Too many requests. IP temporarily blocked.', 429);
});
});
RateLimiter::for('global', function (Request $request) {
return Limit::perMinute(60)->by($request->ip());

app > Http > Middleware > TemporaryIpBlock.php

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Cache;
use Symfony\Component\HttpFoundation\Response;

class TemporaryIpBlock
{
    public function handle(Request $request, Closure $next): Response
    {
        $ip = $request->ip();

        if (Cache::has("blocked_ip:{$ip}")) {
            return response('IP temporarily blocked. Try again later.', 429);
        }

        return $next($request);
    }
}
```

Problems Output Debug Console Terminal Ports

## Attacco fallito (dopo la mitigazione)

Ho rifatto le stesse prove. Dopo le prime richieste la risposta diventa **429** (troppe richieste).

```
> done
200
200
200
200
200
429
429
429
429
429
```